

CENTRO UNIVERSITÁRIO DOM HELDER

CIÊNCIA DA COMPUTAÇÃO

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES I

CAPÍTULO 1

HISTÓRICO, ABSTRAÇÕES E TECNOLOGIAS

COMPUTACIONAIS – Parte I

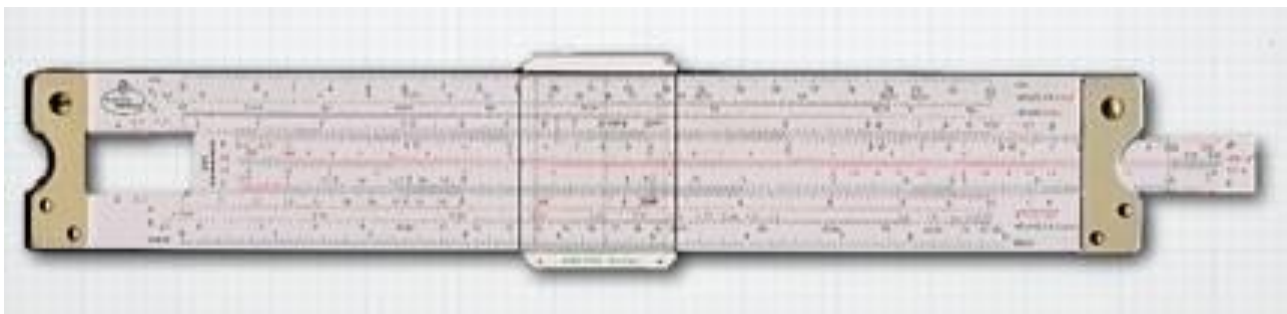
Ábacos

- Muito antes da invenção da calculadora ou do computador, as pessoas realizavam cálculos com um instrumento manual chamado ábaco (abax = tábua de calcular).
- Provavelmente inventado pelo povo sumério, da Mesopotâmia.
- Os egípcios, gregos, romanos, indianos e chineses também usavam o ábaco para fazer contas.



Precursores

- Em 1614, John Napier descobriu uma forma prática de tornar possível transformar multiplicações em somas e divisões em subtração pela disposição de números com uma regra de equivalência.
- Pouco tempo depois, Edmund Gunter conseguiu reduzir ainda mais o esforço, criando um dispositivo onde colocou números em uma linha de uma maneira análoga aos achados de Napier.
- Após uma otimização de William Oughtred, nasceu a régua de cálculo.



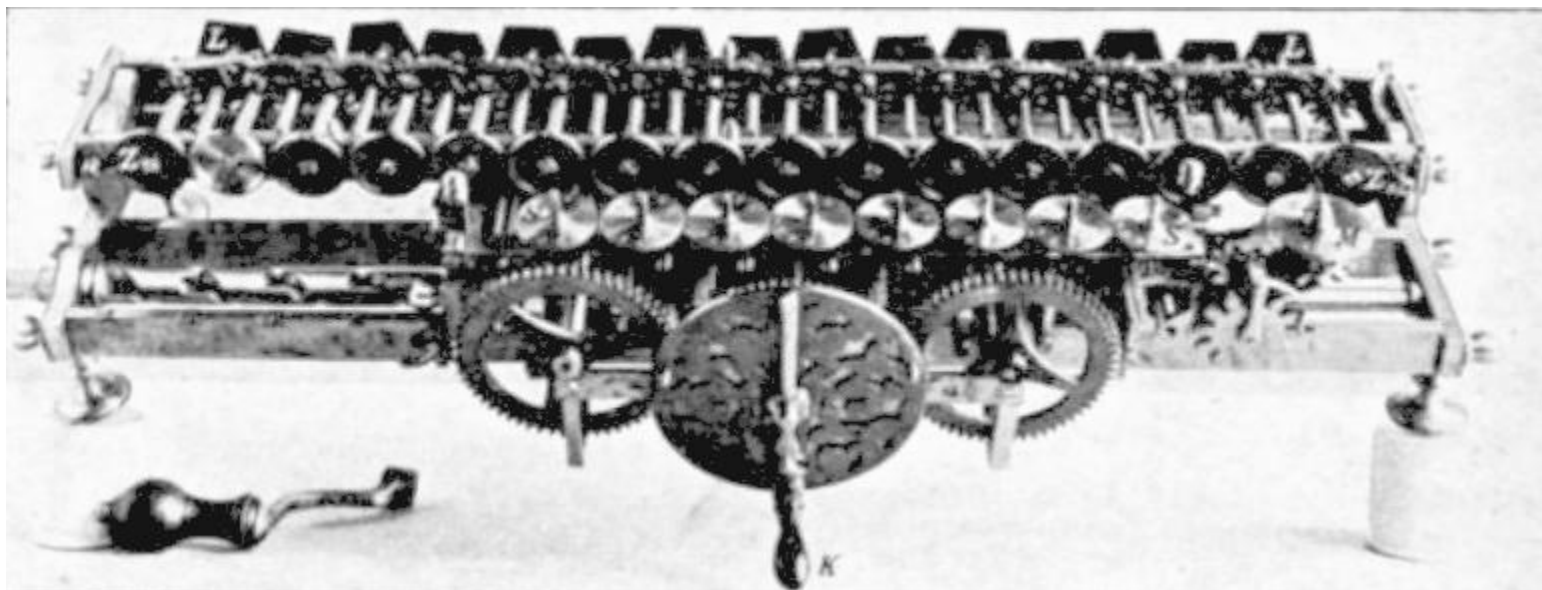
Precursores

- Por volta de 1640, o matemático francês Pascal inventa a primeira máquina de calcular automática (La Pascaline).
- Pascal, com seu conhecimento em física e em matemática, criou uma máquina com um engenhoso sistema de engrenagens que fazia contas de adição e subtração (multiplicação e divisão por repetição).



Precursores

- A primeira calculadora capaz de efetuar os quatro principais cálculos matemáticos, foi criada por Gottfried Wilhelm Leibniz.
- Esse matemático alemão desenvolveu o primeiro sistema de numeração binário moderno que ficou conhecido com "Roda de Leibniz".



Precursores

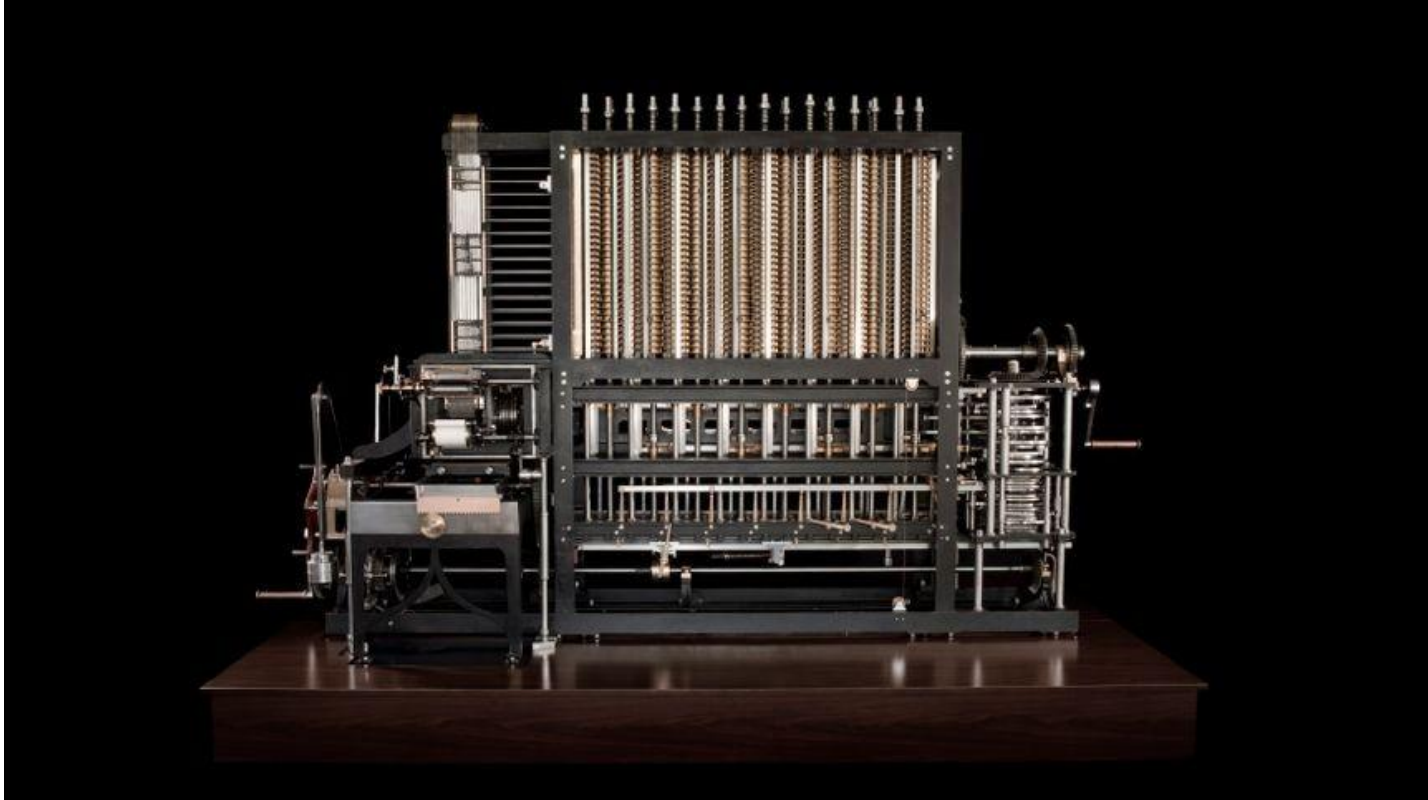
- A primeira máquina mecânica programável foi introduzida pelo matemático francês Joseph-Marie Jacquard.
- Trata-se de um tipo de tear capaz de controlar a confecção dos tecidos através de cartões perfurados.



Precursores

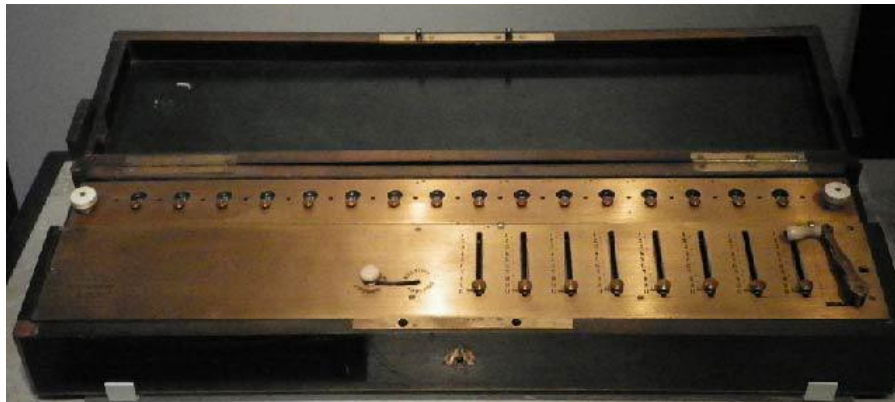
- No século XIX, o matemático inglês Charles Babbage criou uma máquina analítica que, a grosso modo, é comparada com o computador atual com memória e programas.
- Através dessa invenção, alguns estudiosos o consideram o “Pai da Informática”.
- Em 1843, a matemática Lady Ada associou-se a Babbage para o desenvolvimento da máquina analítica.
- Criou o conceito de subrotina, uma sequência de instruções que pode ser usada várias vezes em diferentes contextos.
- Lady Ada é considerada a primeira programadora da história, pois criou os princípios da programação conhecidos como sequência, seleção e repetição.

Precursores



Precursores

- Arithmometer é o nome de um equipamento projetado e patenteado por Charles Xavier Thomas de Colmar no ano de 1820.
- Foi a primeira calculadora construída de forma bem sucedida que permitia somar, subtrair e multiplicar.
- Permitia efetuar divisões, embora com a intervenção do operador.



Precursores

- Samuel Morse, juntamente com Alfred Vail inventam em 1835 o código morse.
- É um sistema de representação de letras, números e sinais de pontuação através de um sinal codificado enviado de forma intermitente.
- Morse também é responsável pela invenção do telégrafo no ano de 1844.



Precursores

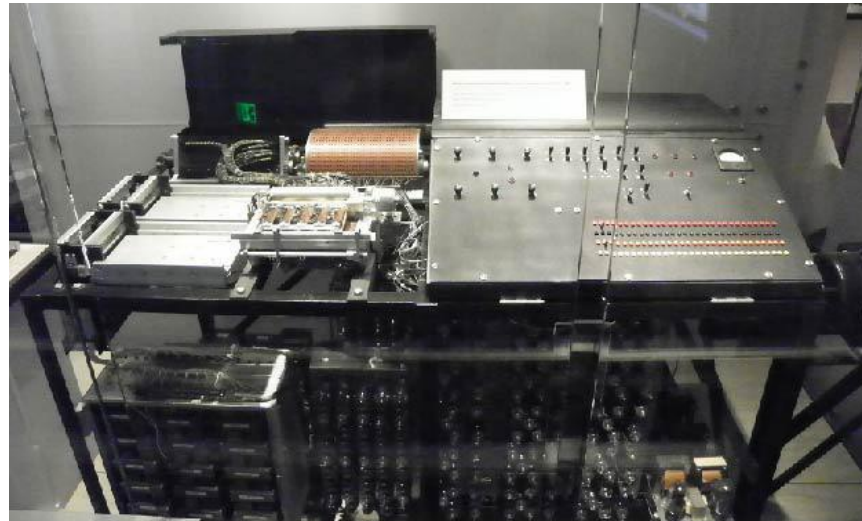
- No ano de 1854, George Boole publica a Álgebra Booleana, cujas ideias constituíram a base da lógica matemática empregada nos computadores.
- A dificuldade de implementar o sistema decimal em componentes elétricos determinaram o uso do sistema binário em computadores.

Ordem	Teoremas	Ordem	Teoremas
1	$A + 0 = A$	11	$A \cdot B + A \cdot B' = A$
2	$A + 1 = 1$	12	$(A + B) \cdot (A + B') = A$
3	$A + A = A$	13	$A + A' \cdot B = A + B$
4	$A + A' = 1$	14	$A \cdot (A' + B) = A \cdot B$
5	$A \cdot 1 = A$	15	$A + B \cdot C = (A + B) \cdot (A + C)$
6	$A \cdot 0 = 0$	16	$A \cdot (B + C) = A \cdot B + A \cdot C$
7	$A \cdot A = A$	17	$A \cdot B + A' \cdot C = (A + C) \cdot (A' + B)$
8	$A \cdot A' = 0$	18	$(A + B) \cdot (A' + C) = A \cdot C + A' \cdot B$
9	$A + A \cdot B = A$	19	$A \cdot B + A' \cdot C + B \cdot C = A \cdot B + A' \cdot C$
10	$A \cdot (A + B) = A$	20	$(A + B) \cdot (A' + C) \cdot (B + C) = (A + B) \cdot (A' + C)$

- Geração 0
- Konrad Zuse (GER) construiu no final da década de 30 uma série de máquinas calculadoras automáticas.
 - Essas máquinas foram destruídas pelo bombardeio em Berlim (1944). Z1: válvulas e Z2: relés.
- John Atanasoff - Universidade de Iowa construiu uma máquina com memória, mas a tecnologia de hardware era inadequada para seu tempo.
- Juntamente com John Berry, iniciou em 1935 um projeto denominado ABC (Atanasoff-Berry Computer).

Histórico

- O ABC era uma máquina dedicada especialmente à solução de conjuntos de equações lineares na Física.
- Funcionava por meio de válvulas, e possuía uma leitora e perfuradora de cartões, o que propiciou o desenvolvimento dos primeiros conceitos que iriam aparecer nos computadores modernos, a unidade de aritmética eletrônica e a memória de leitura e gravação.



Histórico

- O primeiro computador produzido pelos americanos cuja arquitetura era eletromecânica foi o Harvard Mark I que foi desenvolvido em parceria entre a Universidade de Harvard e a IBM.
 - A IBM iniciou em 1935 a produção de calculadoras baseadas em relés. Estas calculadoras produziam em altas velocidades tabelas de vários tipos com alta confiabilidade, conforme havia imaginado Babbage.
- O Harvard Mark I foi desenvolvido pelo matemático americano Howard Hathanway Aiken que atuava como professor na Universidade de Harvard.

Histórico

- Claude Elwood Shannon defendeu, em 1937, a tese de que os conceitos de George Boole teriam grande utilidade no campo da então eletromecânica.
- Ele provou que a álgebra booleana e a aritmética binária poderiam ser utilizadas para simplificar a organização de relés.
- Fez também a analogia inversa, provando a possibilidade de usar organizações de relés para resolver problemas de álgebra booleana.
- Shannon trabalhou com Vannevar Bush e nos Laboratórios Bell.

Histórico

- No ano de 1937 George Stibitz constrói no "Bell Telephone Laboratories" o primeiro calculador binário.
- No ano de 1940 Ainda no "Bell Telephone Laboratories", ele demonstrou o "Complex Number Calculator" (que deve ter sido o primeiro computador digital), em uma conferência no Dartmouth College (Nova York) em 1940.
 - John Mauchley (Professor de física da Universidade da Pennsylvania) estava presente

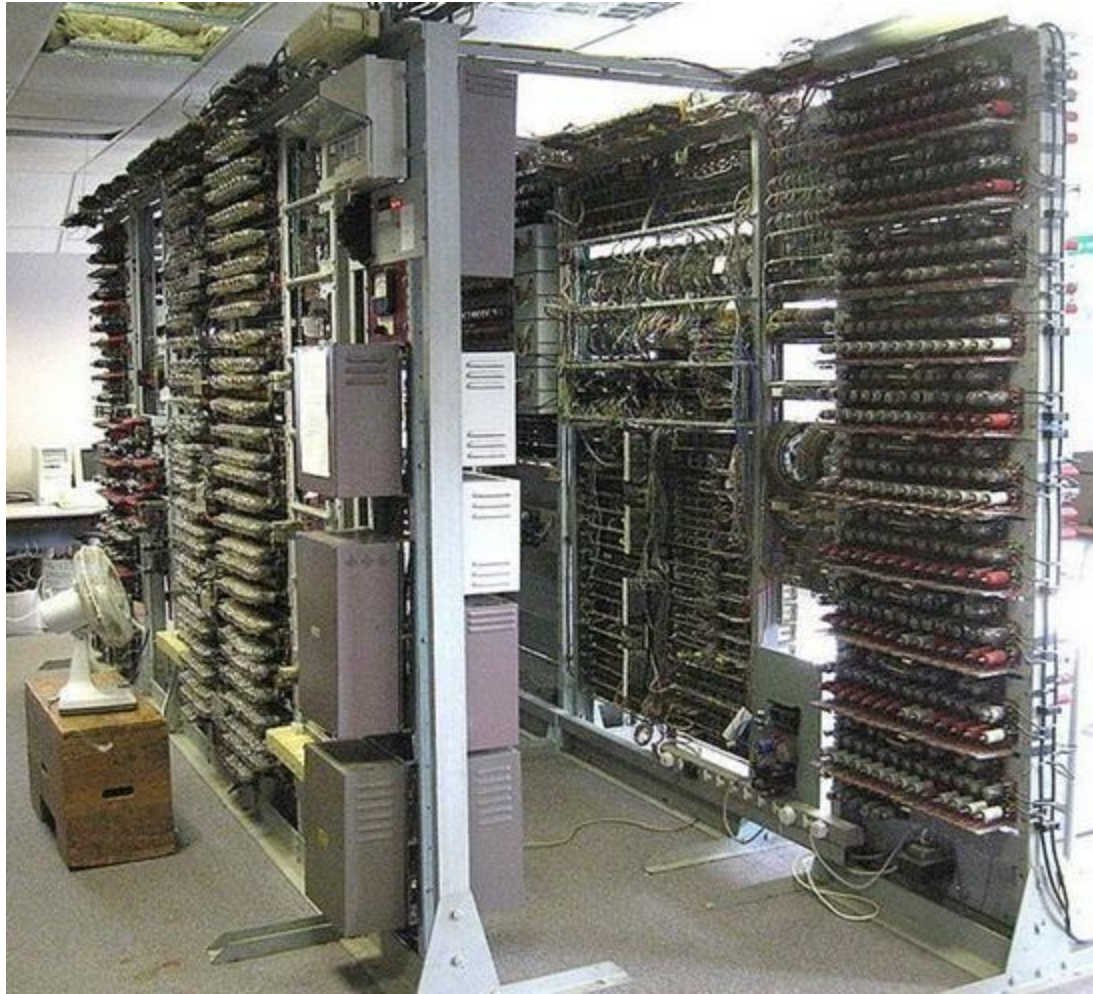
Histórico

- A Segunda Guerra Mundial foi um grande incentivo no desenvolvimento de computadores, visto que as máquinas estavam se tornando mais úteis em tarefas de descriptação de mensagens inimigas e criação de armas mais inteligentes.
- Entre os projetos desenvolvidos no período, os que mais se destacaram foram o Mark I, em 1944, desenvolvido na Universidade Harvard (EUA), e o Colossus, em 1946, criado por Allan Turing.

Histórico

- Sendo uma das figuras mais importantes da computação, Turing focou sua pesquisa na descoberta de problemas formais e práticos que poderiam ser resolvidos por computadores.
- Para aqueles que apresentavam solução, foi criada a famosa teoria da Máquina de Turing, que, com um número finito de operações, resolvia problemas computacionais de diversas ordens.
- A ideia foi colocada em prática com o Colossus.

Histórico



Histórico

- A máquina Lorenz utilizava uma fita de papel que recebia uma sequência de furos que é a “versão visual” dos códigos.
- Ela utilizava um sistema duplo de criptografia: primeiro, a mensagem virava uma sequência de caracteres desconhecidos – nada de letras comuns.
- Em seguida, ela era novamente codificada, embaralhada em uma sequência pseudoaleatória, e só então transmitida via sinais de rádio por um telégrafo.



Histórico

- O codificador funciona a partir da rotação de 12 rodas mecânicas que geram a aleatoriedade (processo com chances limitadas e não tão aleatórias).
- Notar o padrão da sequência (e, posteriormente, do código) era difícil, porém não impossível.
- Após descobrir o funcionamento da Lorenz, os cientistas “só” precisaram construir uma máquina que fizesse o processo reverso mais rapidamente que o trabalho manual.

Histórico

- Colossus: máquina formada por oito grandes “gabinetes” de 2,3 metros de altura, divididas em duas seções de 5,5 metros de comprimento.
- Separadamente, há um leitor com espaço para duas fitas contendo um sinal de rádio impresso em papel.
- Na primeira delas, está a mensagem a ser “traduzida”.
- Na segunda, uma repetição da sequência pseudoaleatória de codificação, que é aplicada à mensagem.

Histórico

- Cada vez que ambas as fitas eram totalmente lidas, a segunda era movida levemente, para que os códigos fossem aplicados com pequenas alterações.
- Cada teste recebia uma “pontuação” – e a rodada de melhor desempenho era aquela verificada pelos engenheiros.
- Em um dos gabinetes fica o painel de controle, em que o algoritmo contendo a sequência pseudoaleatória é configurado manualmente.
- O Colossus não possui um programa instalado, portanto era necessário acertar os pinos e alavancas a cada nova operação.

Histórico

- A leitura era feita a 48 km/h por sensores fotoelétricos – e o primeiro modelo lia incríveis 5 mil caracteres por segundo.
- Por fim, um painel de luzes e uma impressora mostram os resultados obtidos.
- Colossus: reduzia o trabalho de decodificação de semanas para seis horas.
- Ele também não é dotado de memória para armazenar dados.
- Para eliminar esse obstáculo, 2.400 válvulas eletrônicas com fios quentes no centro funcionam como um contador da pontuação obtida pelo Colossus, avisando o próprio sistema se o algoritmo já foi calculado.

Histórico



Histórico

- Primeira Geração (1951-1959)
- Os computadores de primeira geração funcionavam por meio de circuitos e válvulas eletrônicas.
- Possuíam o uso restrito, além de serem imensos e consumirem muita energia.
- John Mauchley sabia que exército americano estava interessado em calculadoras mecânicas.
- Montou proposta solicitando financiamento para a construção de um computador eletrônico
- Proposta aceita em 1943.

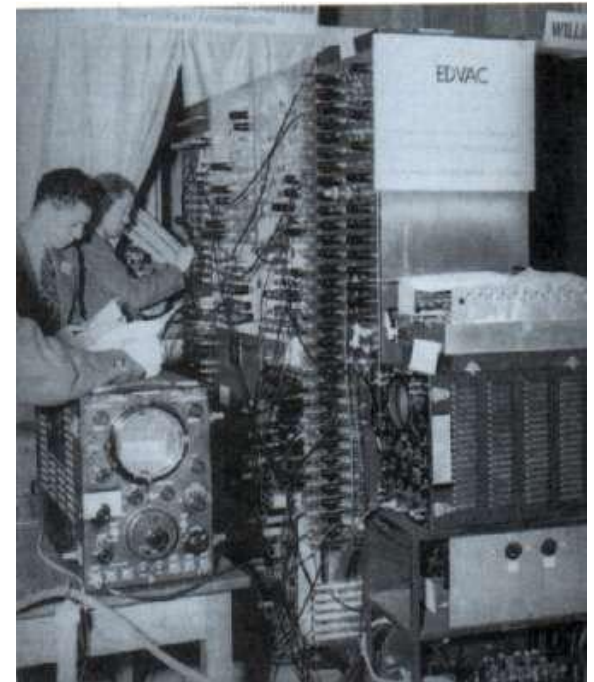
- ENIAC (Eletronic Numerical Integrator And Computer)
- Finalidade: cálculo de artilharia do exército americano
 - Continha 18 mil válvulas
 - 1.500 relés
 - Pesava 30 toneladas
 - Consumia 140 quilowatts de potência
 - Continha 20 registradores com capacidade para conter um número decimal de 10 algarismos.
 - Programado com o ajuste de até 6 mil interruptores e com conexão de uma imensa quantidade de soquetes como uma verdadeira floresta de cabos de jumpers.

Histórico



Histórico

- EDVAC (Eletronic Discrete Variable Automatic Computer)
- Sucessor do ENIAC
- Construído por Mauchley e Eckert
 - Teve seu projeto comprometido porque seus criadores deixaram a Universidade da Pensilvânia para fundar sua própria empresa
 - Eckert e Mauchley Corporation
 - Atualmente UNISYS Corporation



Histórico

- Enquanto isso John Von Neumann (especialista em ciências físicas e matemáticas) que trabalhou no projeto do ENIAC com Mauchley e Eckert constrói sua versão do EDVAC.
- Primeiro computador de programa armazenado
- Herman Goldstine ajudou na elaboração do projeto
 - Programa podia ser em forma digital na memória do computador junto com os dados
 - O projeto básico é conhecido como máquina de Von Neumann – usado no EDSAC.
 - Memória (4096 palavras, uma palavra contendo 40 bits, cada bit um 0 ou 1)
 - Unidade Lógica e Aritmética (ULA)
 - Unidade de Controle (UC)
 - Equipamento de entrada
 - Equipamento de saída

Histórico

- IBM
- Pequena empresa que produzia perfuradoras de cartões e máquinas mecânicas de classificação de cartões
- IBM 701(1953), IBM 704(1956), IBM 709(1958)
- Passa a trabalhar com computadores depois de produzir o IBM 701 em 1953



Histórico

- IBM 701



Histórico

- Segunda Geração (1959-1965)
- Primeiro computador feito com transistores foi o TX-0 (Transistorized eXperimental Computer 0) - MIT



Histórico

- Segunda Geração (1959-1965)



- Ainda com dimensões muito grandes, os computadores da segunda geração funcionavam por meio de transistores, os quais substituíram as válvulas que eram maiores e mais lentas.
- Nesse período já começam a se espalhar o uso comercial.

Histórico

- Logo depois, membro da equipe do TXT fundou a DEC (Digital Equipment Corporation).
- Em 1957 DEC fabrica o PDP-1:
 - Aparece no mercado em 1961
 - 4096 palavras de 18 bits
 - Podia executar 200 mil instruções por segundo
 - Custava 120 mil dólares

Histórico



Histórico

- Em 1957 existiam três grandes linguagens para computadores: APT, FORTRAN e FLOW-MATIC.
- Nesta época era grande a necessidade de uma linguagem simples e padronizada para aplicações comerciais.
- A linguagem que veio para solucionar este problema foi o COBOL (Common Business Oriented Language), tendo sido em grande parte baseada na linguagem FLOW-MATIC que foi desenvolvida por Grace Murray Hopper.

Histórico

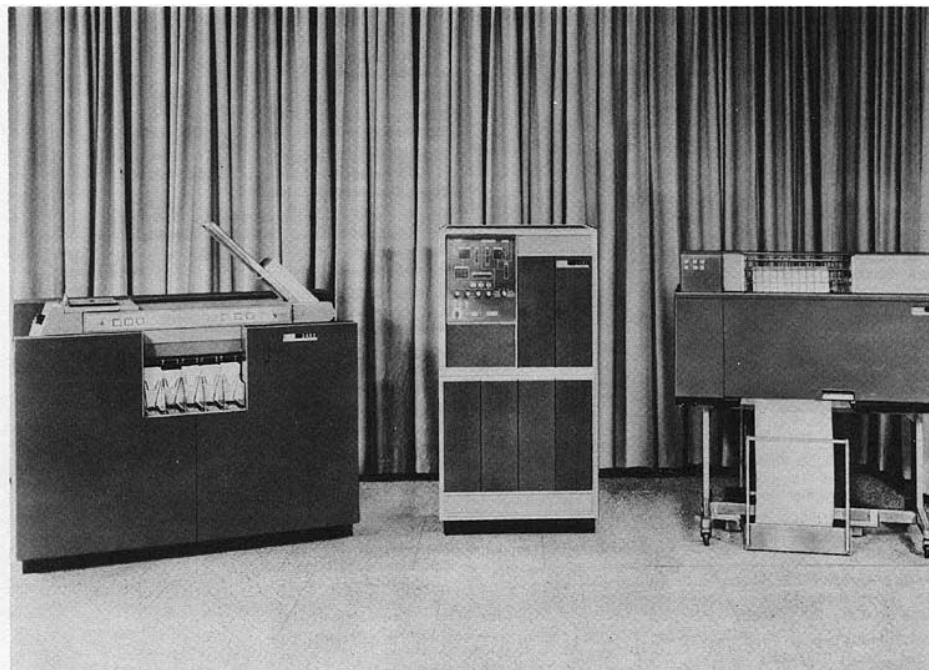
- Anos depois a DEC fabrica o PDP-8:
 - 12 bits
 - Custava 16 mil dólares
 - Tinha barramento único para conectar todos os componentes do computador
 - Empresa vendeu 50 mil computadores



Histórico

- Enquanto isso a IBM lança IBM 7090 e IBM 7094, mas ganha dinheiro com máquina 1401 – destinada a empresas.
- Control Data Corporation (CDC) lança o CDC 6600 – Seymour Cray
- Também surge um computador chamado B5000 - permitia programação em Algol

60



Histórico

- 1958: invenção do circuito integrado de silício
- Permitiu que dezenas de transistores fossem colocados em um único chip
- Computador menor, mais rápido, mais barato
- 1964: IBM lança linha System/360 para substituir suas duas máquinas precursoras (IBM 7094 e IBM 1401)



- Linha System/360
- Modelo 195
 - Permitiu a multiprogramação
 - Primeira máquina que podia emular (simular a execução) outros computadores
 - Seguido pelas séries 370, 4300, 3080, 3090

Histórico

- Terceira Geração (1965-1975)
- Os computadores da terceira geração funcionavam por circuitos integrados.
- Esses encapsularam transistores e já apresentavam uma dimensão menor e maior capacidade de processamento.
- Foi nesse período que os chips foram criados e a utilização de computadores pessoais começou.



Histórico

- A DEC lança série PDP-11 – atualmente HP
- Sucessor de 16 bits do PDP-8.
- Memória semicondutora (MOS), em vez de núcleo.
- Havia três ou quatro gabinetes de tamanho normal: CPU, memória, disco de troca de cabeça fixa (2K),;
- Comunicações com (32 linhas seriais DH11)



Histórico

- Quarta Geração (1975 - até os dias atuais)
- Com o desenvolvimento da microeletrônica os computadores diminuem de tamanho, aumentam a velocidade e capacidade de processamento de dados.
- Década de 1980
 - VLSI (Very Large Scale Integration – Integração em escala muito grande)
 - Possibilitou colocar milhões de transistores em um único chip
 - Computador mais rápido e menor
 - Início da era do Computador Pessoal



Histórico

- 1981 A IBM lança seu computador pessoal.



Histórico

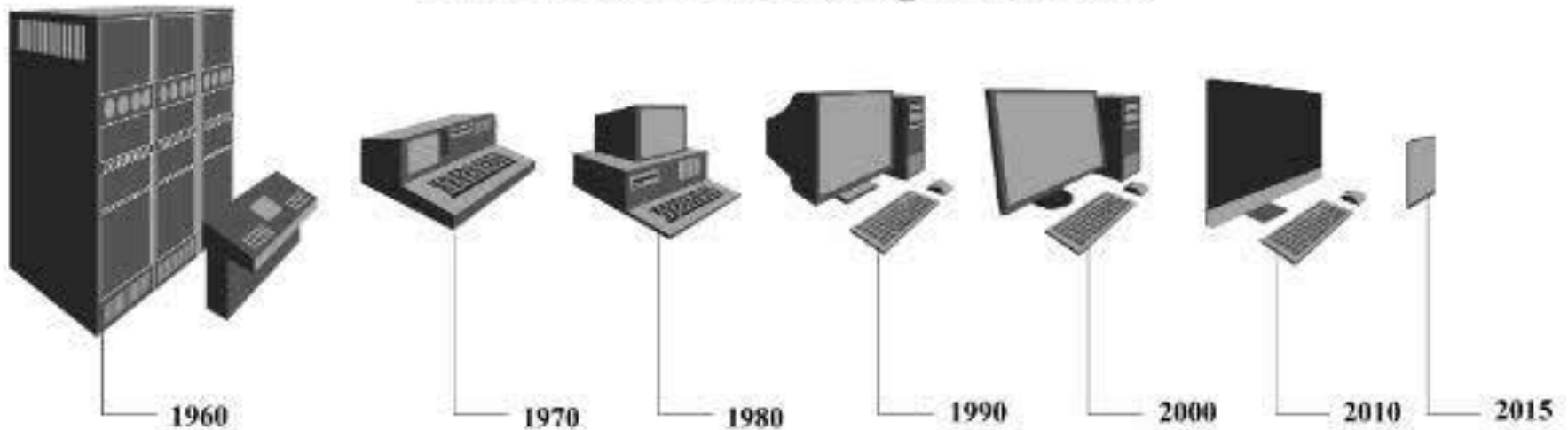
- 1984 – Apple lança Mac Intosh
- Lançou anos antes o Apple e o Apple II
- Mercado leva ao desejo de computadores portáteis
- Surge o Osborne-1 com 11 kg



Histórico

- 1985 – Intel lança o 386 – 1º. Pentium
- Até 1992, os computadores pessoais eram de 8, 16 ou 32 bits
- DEC lança Alpha com 64 bits

Evolução dos Computadores



Introdução

- Existe uma lacuna entre o que é conveniente para as pessoas e o que é conveniente para computadores:
 - pessoas querem fazer X
 - computadores só podem fazer Y
- Isso gera um grande problema!
- Um dos objetivos deste curso é explicar como esse problema pode ser resolvido.

Introdução

- Duas abordagens
 - ambas envolvem projetar um conjunto de instruções que é mais conveniente para as pessoas usarem do que o conjunto embutido de instruções de máquina.
- As instruções para as pessoas formam uma linguagem (L1), assim como as instruções de máquina embutidas formam uma linguagem (L0).
- As duas técnicas diferem no modo como os programas escritos em L1 são executados pelo computador que, afinal, só pode executar programas escritos em sua linguagem de máquina, L0.

Introdução

- Um método de execução de um programa escrito em L1 é primeiro substituir cada instrução nele por uma sequência equivalente de instruções em L0.
- O programa resultante consiste totalmente em instruções L0.
- O computador executa o novo programa L0 em vez do antigo programa L1.
- Essa técnica é chamada de tradução.

Introdução

- A outra técnica é escrever um programa em L0 que considere os programas em L1 como dados de entrada e os execute, examinando cada instrução por sua vez, executando diretamente a sequência equivalente de instruções L0.
- Essa técnica não requer que se gere um novo programa em L0.
- Ela é chamada de interpretação, e o programa que a executa é chamado de interpretador.
- Tradução e interpretação são semelhantes.
- Nos dois métodos, o computador executa instruções em L1 executando sequências de instruções equivalentes em L0.

Introdução

- Na tradução, o programa L1 inteiro primeiro é convertido para um L0, o programa L1 é desconsiderado e depois o novo L0 é carregado na memória do computador e executado.
- Durante a execução, o programa L0 recém gerado está sendo executado e está no controle do computador.
- Na interpretação, depois que cada instrução L1 é examinada e decodificada, ela é executada de imediato (nenhum programa traduzido é gerado).
- Aqui, o interpretador está no controle do computador. Para ele, o programa L1 é apenas de dados.
- Ambos os métodos (e cada vez mais, uma combinação dos dois) são bastante utilizados.

Introdução

- Em vez de pensar em termos de tradução ou interpretação, muitas vezes é mais simples imaginar a existência de um computador hipotético ou máquina virtual cuja linguagem seja L1.
- Vamos chamar essa máquina virtual de M1 (e de M0 aquela correspondente a L0).
- Se essa máquina pudesse ser construída de forma barata o suficiente, não seria preciso de forma alguma ter a linguagem L0 ou uma máquina que executou os programas em L0.
- As pessoas poderiam simplesmente escrever seus programas em L1 e fazer com que o computador os executasse diretamente.
- Mesmo que a máquina virtual cuja linguagem é L1 seja muito cara ou complicada de construir com circuitos eletrônicos, as pessoas ainda podem escrever programas para ela.

Introdução

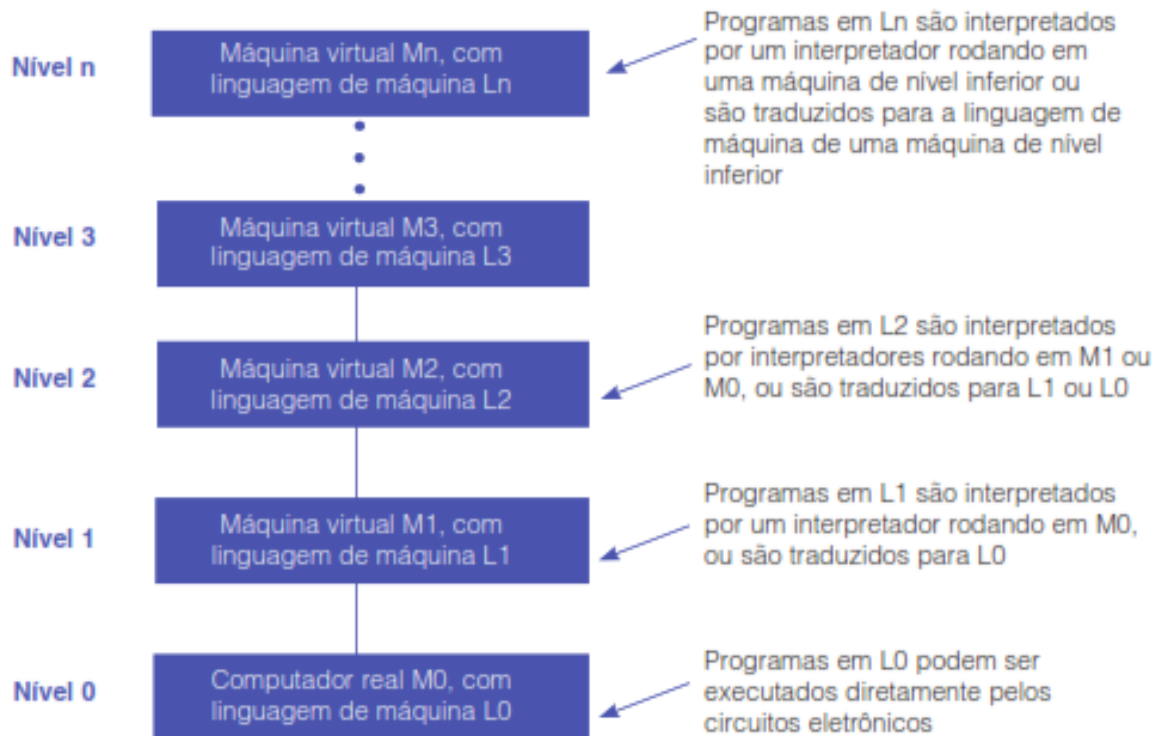
- Esses programas podem ser ou interpretados ou traduzidos por um programa escrito em LM que, por si só, consegue ser executado diretamente pelo computador real.
- Em outras palavras, as pessoas podem escrever programas para máquinas virtuais, como se realmente existissem.
- Para tornar prática a tradução ou a interpretação, as linguagens L0 e L1 não deverão ser “muito” diferentes.
- Tal restrição significa quase sempre que L1, embora melhor que L0, ainda estará longe do ideal para a maioria das aplicações.
- Esse resultado talvez seja desanimador à luz do propósito original da criação de L1 – aliviar o trabalho do programador de ter que expressar algoritmos em uma linguagem mais adequada a máquinas do que a pessoas.

Introdução

- Abordagem óbvia: inventar um conjunto de instruções que seja mais orientado a pessoas e menos orientado a máquinas que a L1.
- Esse terceiro conjunto também forma uma linguagem: L2 (com a máquina virtual M2).
- As pessoas podem escrever programas em L2 exatamente como se de fato existisse uma máquina real com linguagem de máquina L2.
- Esses programas podem ser traduzidos para L1 ou executados por um interpretador escrito em L1.
- A invenção de toda uma série de linguagens, cada uma mais conveniente que suas antecessoras, pode prosseguir indefinidamente, até que, por fim, se chegue a uma adequada.
- Cada linguagem usa sua antecessora como base, portanto, podemos considerar um computador que use essa técnica como uma série de camadas ou níveis.

Introdução

- Máquina Multinível



Introdução

- De certa forma, um computador com n níveis pode ser visto como n diferentes máquinas virtuais, cada uma com uma linguagem de máquina diferente.
- Usaremos os termos “nível” e “máquina virtual” para indicar a mesma coisa.
- Apenas programas escritos na linguagem L_0 podem ser executados diretamente pelos circuitos eletrônicos, sem a necessidade de uma tradução ou interpretação intervenientes.
- Os programas escritos em $L_1, L_2, \dots, L_n$ devem ser interpretados por um interpretador rodando em um nível mais baixo ou traduzidos para outra linguagem correspondente a um nível mais baixo.

Introdução

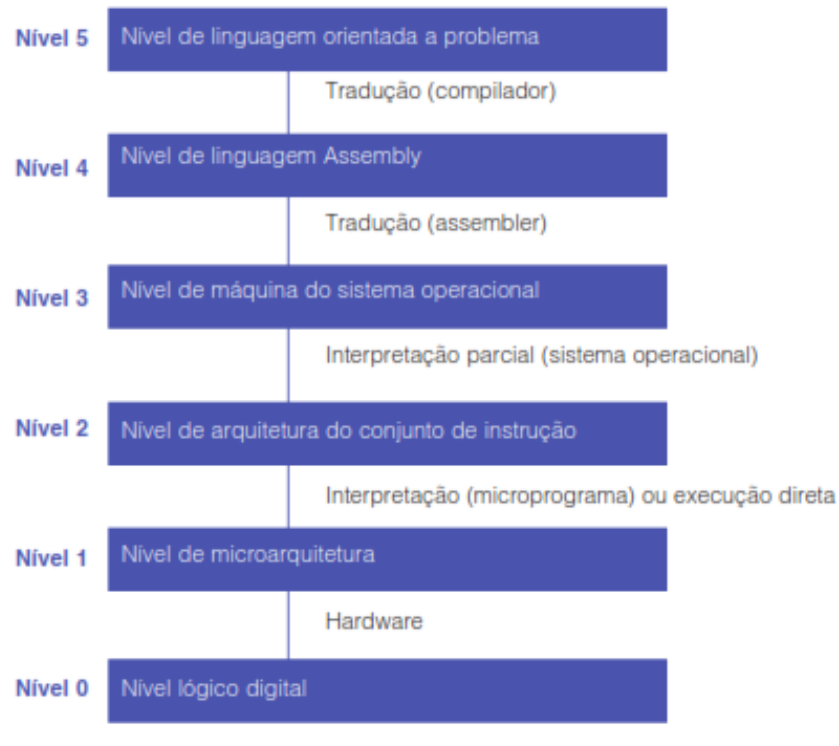
- Uma pessoa que escreve programas para a máquina virtual de nível n não precisa conhecer os interpretadores e tradutores subjacentes.
- A estrutura de máquina garante que esses programas, de alguma forma, serão executados.
- Não há interesse real em saber se eles são executados passo a passo por um interpretador que, por sua vez, também é executado por outro interpretador, ou se o são diretamente pelos circuitos eletrônicos.
- O mesmo resultado aparece nos dois casos: os programas são executados.
- Quase todos os programadores que usam uma máquina de nível n estão interessados apenas no nível superior, aquele que menos se parece com a linguagem de máquina do nível mais inferior.

Introdução

- Porém, as pessoas interessadas em entender como um computador realmente funciona deverão estudar todos os níveis.
- Quem projeta novos computadores ou novos níveis também deve estar familiarizado com outros níveis além do mais alto.

Introdução

- Computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome que o suporta)



Introdução

- Por que estudar arquitetura de computadores?
 - sempre foi importante projetar sistemas de computação visando alto desempenho;
- Características de desempenho básicas dos sistemas computacionais:
 - velocidade de processador
 - velocidade de memória
 - capacidade de memória e
 - taxas de dados de interconexão
- Dificuldade de projetar um sistema balanceado, que maximize o desempenho e a utilização de todos os elementos.
- Técnica: mudar a estrutura ou a função em uma área para compensar uma divergência de desempenho em outra.

Classes de Computadores

- Desktop
 - Enfatizam o bom desempenho a um único usuário por um baixo custo e normalmente são usados para executar software independente.
- Servidor
 - Usado para executar grandes programas para múltiplos usuários, quase sempre de maneira simultânea e normalmente acessado apenas por meio de uma rede.
- Supercomputador
 - Classe de computadores com desempenho e custo mais altos; eles são configurados como servidores.
 - ✓ Clusters de Computadores: warehouse
 - ✓ Paralelismo

Aplicações

- IoT:
 - Computadores embarcados (dentro de outros) que estão conectados à Internet, geralmente sem fio. Quando aumentados com sensores e atuadores, os dispositivos IoT coletam dados úteis e interagem com o mundo físico, levando a uma grande variedade de aplicações “inteligentes”.
- Dispositivo pessoal móvel (PMD)
 - Dispositivos sem fio com interfaces de usuário multimídia
 - Custo é a principal preocupação
 - Eficiência energética
 - Aplicativos para PMDs muitas vezes são baseados na web e orientados para a mídia
 - Processadores em um PMD considerados computadores embarcados

Aplicações

- São plataformas que podem executar software desenvolvido externamente e compartilham muitas das características dos desktop.
- Capacidade de executar softwares de terceiros como a linha divisória entre computadores não embarcados e embarcados.
- A capacidade de resposta e previsibilidade são características-chave para aplicações de mídia. Um requisito de desempenho em tempo real significa que um segmento da aplicação tem um tempo absoluto máximo de execução.

Abaixo do seu programa...

- Os computadores são escravos de nossos comandos $\Rightarrow$ INSTRUÇÃO.
- Exemplo: 1000110010100000 $\Rightarrow$ adição.
- Tanto as instruções quanto os números são representados por conjuntos de dígitos binários.
- Início:
 - $\Rightarrow$ programas eram escritos em binário
 - $\Rightarrow$ baixa produtividade e grande quantidade de erros!
- 1ª evolução $\Rightarrow$ linguagem de montagem $\Rightarrow$ uso da máquina para programar a própria máquina.
- Ex.: add A, B $\Rightarrow$ montador $\Rightarrow$ 1000110010100000

Abaixo do seu programa...

- Problema da linguagem de montagem: distante da notação com a qual o ser humano está acostumado a raciocinar.
- 2ª evolução: linguagem de alto nível
- Ex.: $A + B \Rightarrow$ compilador $\Rightarrow$ add A, B $\Rightarrow$ montador 1000110010100000
- Vantagens da linguagem de alto nível:
 - Permitem raciocínio mais próximo do natural do ser humano;
 - São projetadas de acordo com o programa a ser desenvolvido;
 - Aumento da produtividade do programador;
 - Programas tornam-se independentes do hardware;
 - Sanidade mental dos programadores.

Abaixo do seu programa...

- O que é interessante ressaltar:
- O uso da máquina para programar a própria máquina chegou a um nível de abstração tão grande que as máquinas atuais já possuem autonomia até para definir procedimentos corretos ou não.
 - A máquina controla o funcionamento da própria máquina: SO;
 - O sistema operacional é um programa que não tem “nada” de diferente do seu programa?
- Futuro:
 - Máquinas capazes de se autoprogramar.
 - Máquinas capazes de se reproduzir.

O que é um computador?

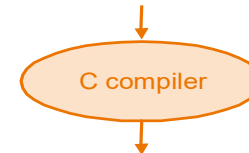
- Abstração: o processador é um ULSI (implementado usando milhões/bilhões de transistores).
- Impossível de entender olhando cada componente isoladamente.
- Blocos funcionais (caixas)
 - Manter os níveis de detalhes mais baixos escondidos
 - Ex.: a memória é construída com flip-flops, que são construídos com portas lógicas, que são construídas com transistores.

Abstração

- Maior aprofundamento revela mais informações.
- Uma abstração omite detalhes desnecessários, ajudando a lidar com a complexidade do sistema.

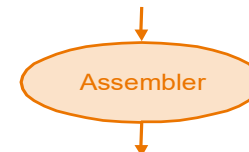
High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```



Assembly
language
program
(for MIPS)

```
swap:
  muli $2, $5, 4
  add $2, $4, $2
  lw $15, 0($2)
  lw $16, 4($2)
  sw $16, 0($2)
  sw $15, 4($2)
  jr $31
```

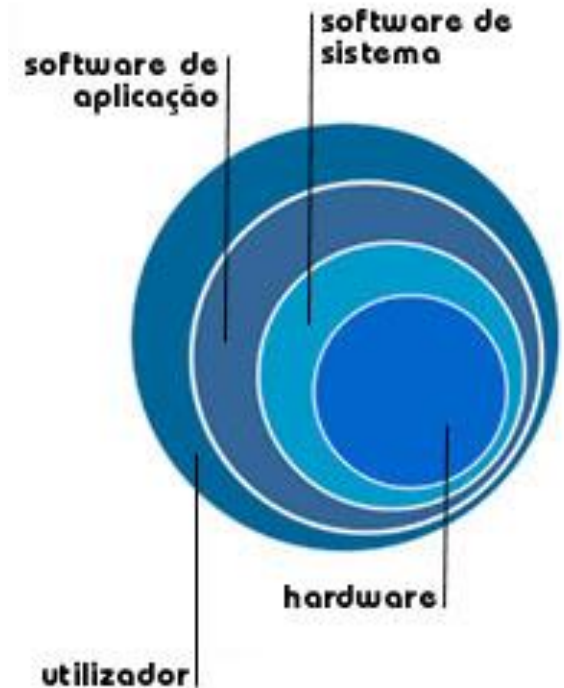


Binary machine
language
program
(for MIPS)

```
000000001010000100000000000011000
00000000100011100001100000100001
10001100011000100000000000000000
100011001111001000000000000000100
10101100111100100000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```


Por trás do Programa

- Software de sistemas: é o responsável pela gestão de todos os componentes do computador. Fornece serviços que normalmente são úteis, incluindo SsOs, compiladores e montadores.
- Software de aplicação: em regra é criado para executar tarefas específicas (processamento de texto, reprodução de áudio, etc).
- Ao contrário do software de sistema, estas tarefas não são indispensáveis ao normal funcionamento do computador (são executadas a pedido do utilizador).



Por trás do Programa

- O SO fornece a interface entre programa de usuário / hardware e disponibiliza diversos serviços e funções de supervisão.
- Entre as funções mais importantes estão:
 - Manipular as operações básicas de entrada e saída;
 - Alocar armazenamento e memória;
 - Possibilitar e controlar o compartilhamento do computador entre as diversas aplicações que o utilizam simultaneamente.

Mais abstração....

- Uma das abstrações mais importantes é a ARQUITETURA DO CONJUNTO DE INSTRUÇÕES (ISA) ou simplesmente CONJUNTO DE INSTRUÇÕES DE UMA MÁQUINA.
 - É a interface entre hardware e software de baixo nível.
 - É o conjunto de instruções que podem ser utilizadas em determinada máquina.
- Vantagem: admite diferentes implementações da mesma arquitetura.
- Desvantagem: algumas vezes impossibilita o uso de inovações.
- Essa interface abstrata permite que muitas implementações com custo e desempenho variáveis executem um software idêntico.

Arquitetura do Conjunto

- A evolução das máquinas está baseada em dois campos:
 - Melhorias com relação aos materiais de fabricação
 - desempenho do hardware → microeletrônica.
 - Melhorias na organização do funcionamento do computador:
 - Modos de execução das instruções.
 - Paralelismo.
 - Divisão da memória em níveis hierárquicos.
 - Utilização de memória cache.

Definições de Memórias

- A ambiguidade 2^x versus 10^y bytes foi resolvida acrescentando-se uma notação binária para todos os termos de tamanho comuns.
- Na última coluna, observamos como o termo binário é maior do que seu correspondente decimal, o que é visto quando descemos na tabela.
- Esses prefixos funcionam para bits e também como bytes, portanto gigabit (Gb) é 10^9 bits, enquanto gibibits (Gib) é 2^{30} bits.

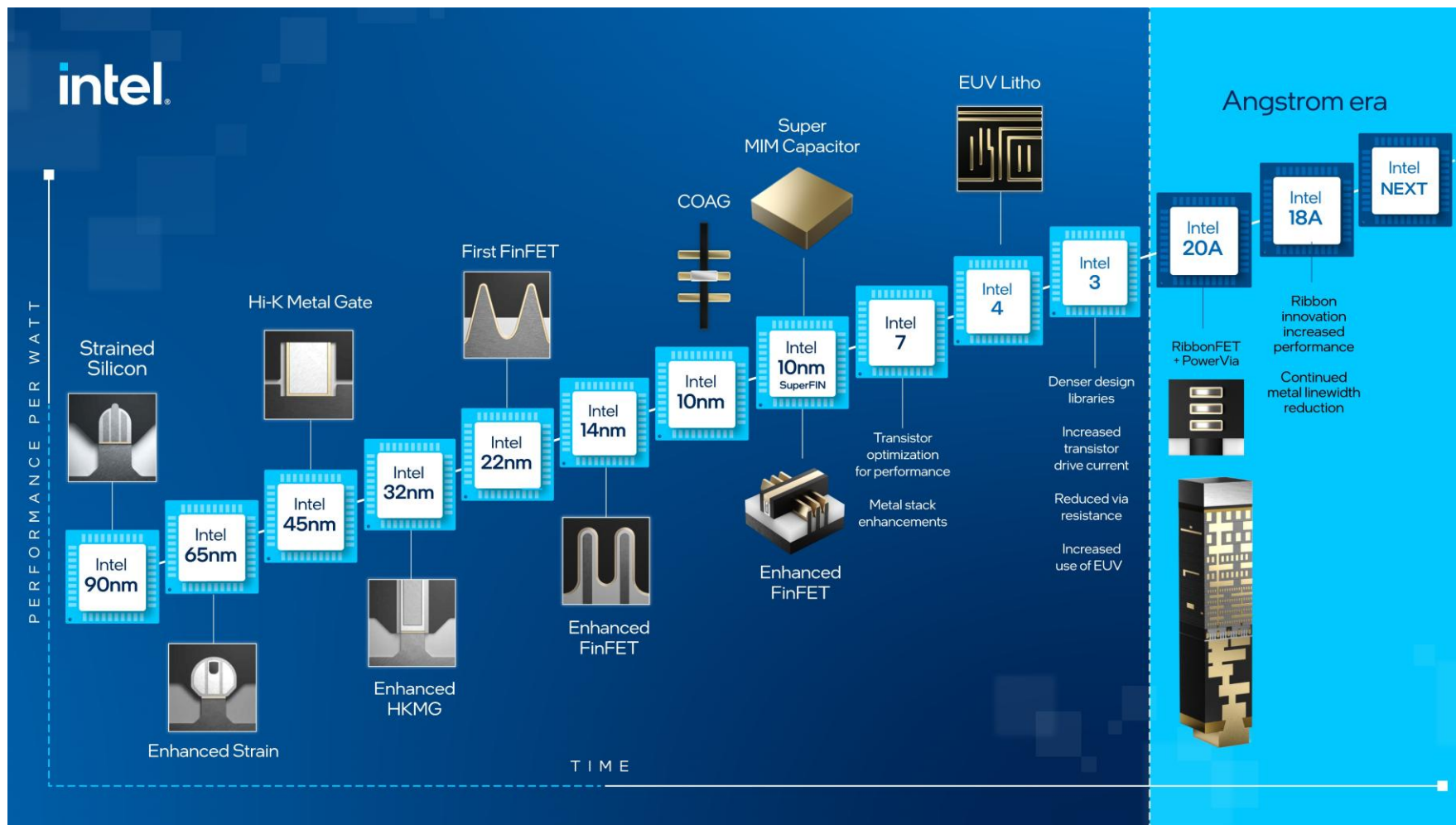
Termo decimal	Abreviação	Valor	Termo binário	Abreviação	Valor	% maior
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
zottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%

Oito Ideias

1. Projete pensando na “Lei” de Moore

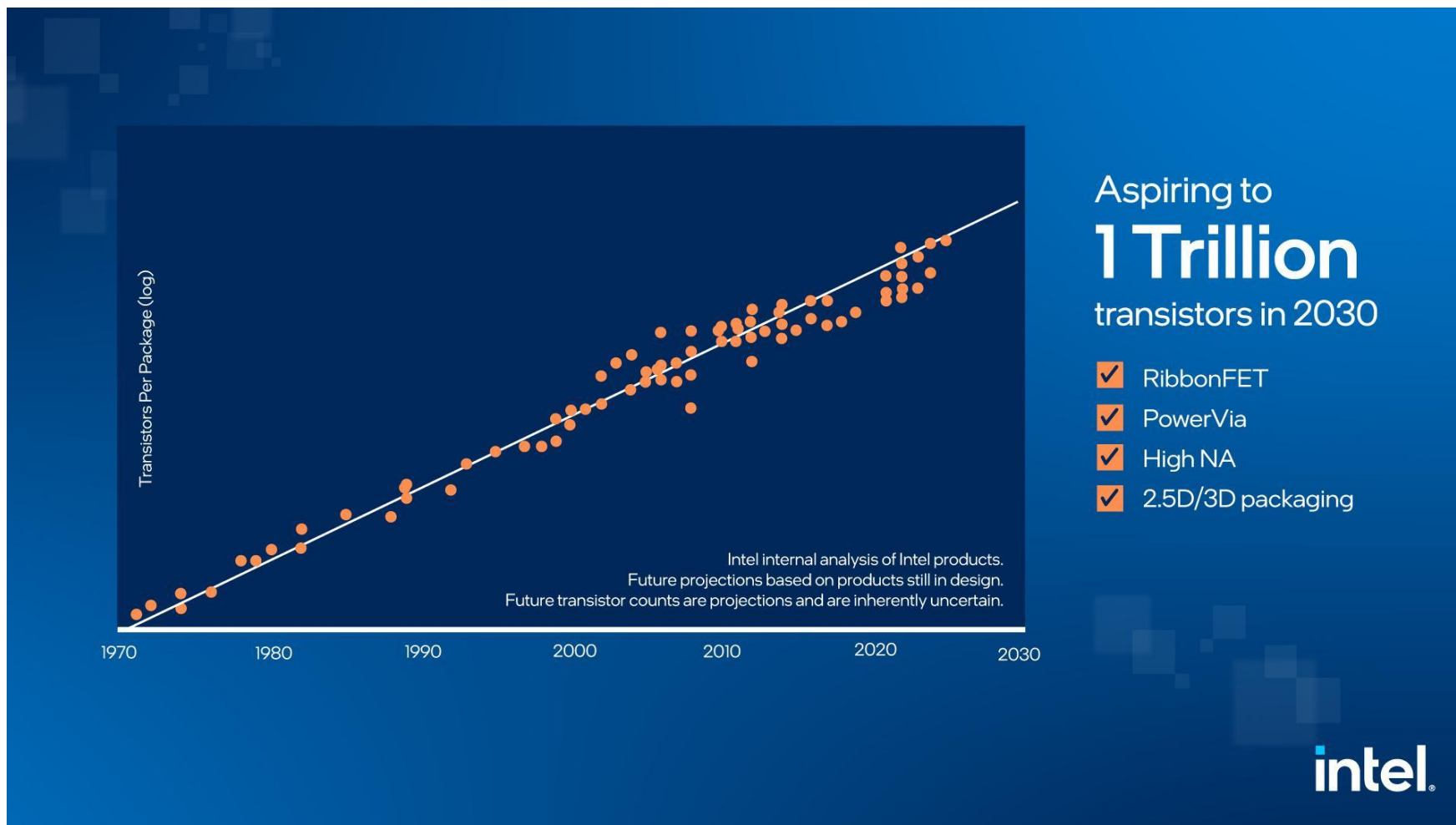
- A única constante para os projetistas de computador é a mudança rápida, que é controlada em grande parte pela Lei de Moore.
- Ela declara que os recursos do circuito integrado dobram a cada 18 a 24 meses.
- A Lei de Moore resultou de uma previsão desse crescimento na capacidade do CI em 1965, por Gordon Moore, um dos fundadores da Intel.
- Como os projetos de computador podem durar anos, os recursos disponíveis por chip podem facilmente dobrar ou quadruplicar entre o início e o final do projeto.

“Lei” de Moore



[Lei de Moore – Agora e no Futuro \(intel.com.br\)](https://www.intel.com.br)

“Lei” de Moore



Lei de Moore: número de transistores por dispositivo: passado, presente, futuro

2. Use a abstração para simplificar o projeto

- Os arquitetos e os programadores de computador tiveram que inventar técnicas para que se tornassem mais produtivos, pois, de outra forma, o tempo de projeto aumentaria de modo insustentável quando os recursos aumentassem pela Lei de Moore.
- Uma técnica de produtividade importante para o hardware e o software é usar abstrações para representar o projeto em diferentes níveis de representação; os detalhes de nível mais baixo serão ocultados, para oferecer um modelo mais simples nos níveis mais altos.

Oito Ideias

3. Torne o caso comum veloz

- Tornar o caso comum veloz costuma melhorar mais o desempenho do que otimizar o caso raro.
- Ironicamente, o caso comum normalmente é mais simples do que o caso raro e, portanto, geralmente é mais fácil de melhorar.
- Esse conselho do senso comum implica que você saberá qual é o caso comum, o que é possível apenas com experimentação e medição cuidadosas.

Oito Ideias

4. Desempenho pelo paralelismo

- Desde o nascimento da computação, os arquitetos de computador tem oferecido projetos que geram mais desempenho realizando operações em paralelo.

5. Desempenho pelo Pipelining

- Um padrão de paralelismo em particular é tão prevalecente na arquitetura de computação que merece seu próprio nome: pipelining.

6. Desempenho pela predição

- Seguindo o ditado de que pode ser melhor pedir perdão do que pedir permissão, a última grande ideia a é predição.
- Em alguns casos, na média, pode ser mais rápido prever e começar a trabalhar do que esperar até que você saiba ao certo, supondo que o mecanismo para se recuperar de um erro de previsão não seja tão dispendioso e sua predição seja relativamente precisa.

7. Hierarquia de memórias

- Os programadores desejam que a memória seja rápida, grande e barata, pois a velocidade da memória geralmente modela o desempenho, a capacidade limita o tamanho dos problemas que podem ser resolvidos e o custo da memória, hoje, geralmente é o maior custo do computador.
- Os arquitetos descobriram que podem resolver essas demandas em conflito com uma hierarquia de memórias, com a memória mais rápida, menor e mais cara por bit no topo da hierarquia e a mais lenta, maior e mais barata por bit no fundo.

8. Estabilidade pela redundância

- Os computadores não apenas precisam ser rápidos; eles precisam ser estáveis.
- Como qualquer dispositivo pode falhar, montamos sistemas estáveis incluindo componentes redundantes, que podem assumir o controle quando uma falha ocorre e ajudar a detectá-la.